REVISION CHEAT SHEET: ABSTRACTION / INTERFACES

Abstraction and Interfaces in C#

Interview-ready notes · examples · C# · diagrams

Topic 15 of 40

Abstraction is one of the four major object-oriented programming principles:

EXAMPLE / FLOW

```
Encapsulation

Inheritance

Polymorphism

Abstraction
```

The central idea of abstraction is:

> **Expose what a component can do without forcing its caller to understand every detail of how it does it.**

Consider your Fundraising Platform.

A service may need to save a gift:

C#

```
await repository.SaveAsync(
    gift,
    cancellationToken);
```

The caller should not need to understand:

EXAMPLE / FLOW

```
SQL connection creation

EF Core DbContext

INSERT statements

Transactions

Retry logic

Database provider details
```

The caller only needs to know:

EXAMPLE / FLOW

```
IGiftRepository
    →
SaveAsync(...)
```

That is abstraction.

High-Level Mental Model

EXAMPLE / FLOW

```
Caller
    →
Abstraction
    →
Implementation
```

For example:

EXAMPLE / FLOW

```
GiftService
    →
IGiftRepository
    →
SqlGiftRepository
    →
SQL Server
```

The caller depends on:

EXAMPLE / FLOW

```
WHAT must happen
```

rather than:

EXAMPLE / FLOW

```
HOW it happens
```

Topic Coverage

This topic covers:

- what abstraction means;
- abstraction vs encapsulation;
- abstract classes;
- abstract methods;
- abstract properties;
- abstract-class constructors;
- concrete vs abstract classes;
- interfaces;
- implementing interfaces;
- implicit interface implementation;
- explicit interface implementation;
- interface properties;
- interface methods;
- interface events and indexers;
- multiple interfaces;
- interface inheritance;
- abstract class vs interface;
- interface polymorphism;
- default interface implementations;

- modern interface access rules;
- static interface members;
- `static abstract`;
- `static virtual`;
- generic static polymorphism;
- interfaces with dependency injection;
- interface segregation preview;
- repository/service contracts;
- common mistakes;
- performance considerations;
- security considerations;
- testing;
- ASP.NET Core usage;
- Fundraising Platform examples;
- interview questions.

What Is Abstraction?

Definition

Abstraction means presenting the essential capabilities of something while hiding unnecessary implementation details from the caller.

Imagine driving a car.

You use:

```
EXAMPLE / FLOW
Steering Wheel

Brake

Accelerator
```

You don't normally need to understand:

```
EXAMPLE / FLOW
Fuel injection calculations

Transmission internals

Engine timing

Brake hydraulic pressure
```

The car exposes an abstraction.

Software works similarly.

Software Example

A caller sees:

```
C#
public interface IGiftRepository
{
    Task SaveAsync(
        Gift gift,
```

```
        CancellationToken cancellationToken);
}
```


It doesn't need to know whether the implementation uses:

EXAMPLE / FLOW

```
EF Core

ADO.NET

Dapper

In-memory storage

External service
```

Abstraction Flow

EXAMPLE / FLOW

```
Complex Internal Implementation
        ↓
Expose Essential Contract
        ↓
Caller Uses Contract
        ↓
Implementation Can Change
```

Why Abstraction Exists

Without abstraction:

EXAMPLE / FLOW

```
GiftService
    →
SQL Connection

GiftService
    →
SQL Commands

GiftService
    →
Transaction Logic

GiftService
    →
Database Provider
```

The service becomes tightly coupled to persistence details.

With abstraction:

EXAMPLE / FLOW

```
GiftService
    →
IGiftRepository
```

Implementation details live elsewhere:

EXAMPLE / FLOW

```
IGiftRepository
        ↓
SqlGiftRepository
        ↓
Database
```

Abstraction Reduces Coupling

Suppose:

C#

```
public sealed class GiftService
{
    private readonly SqlGiftRepository
        _repository;

    public GiftService(
        SqlGiftRepository repository)
    {
        _repository = repository;
    }
}
```

`GiftService` is directly coupled to:

EXAMPLE / FLOW

```
SqlGiftRepository
```

If the implementation changes to:

EXAMPLE / FLOW

```
CachedGiftRepository

CosmosGiftRepository

FakeGiftRepository
```

the service may need modification.

Depending on an Abstraction

Instead:

C#

```
public sealed class GiftService
{
    private readonly IGiftRepository
        _repository;

    public GiftService(
        IGiftRepository repository)
    {
        _repository = repository;
    }
}
```

Now:

EXAMPLE / FLOW

```
GiftService
    →
IGiftRepository
```

The implementation can vary.

Abstraction vs Encapsulation

These concepts are related but different.

Abstraction

Focuses on:

EXAMPLE / FLOW

```
WHAT does this component provide?
```

Example:

C#

```
repository.SaveAsync(...);
```

Caller sees the operation.

It does not care about SQL internals.

Encapsulation

Focuses on:

EXAMPLE / FLOW

```
HOW is internal state protected?
```

Example:

C#

```
public decimal TotalAmount
{
    get;
    private set;
}
```

External callers cannot freely change:

EXAMPLE / FLOW

```
TotalAmount
```

Comparison

Abstraction	Encapsulation
Hides unnecessary complexity	Protects internal state/details
Focuses on what	Focuses on controlled access/how
Often interfaces/abstract classes	Often private fields/properties/methods
Reduces coupling	Protects invariants

Abstraction	Encapsulation
Exposes essential contract	Restricts internal manipulation

Example Combining Both

```C#
public interface IGiftProcessor
{
    void Process(
        Gift gift);
}
```

This is:

```EXAMPLE / FLOW
Abstraction
```

Implementation:

```C#
public sealed class GiftProcessor :
    IGiftProcessor
{
    private readonly IGiftRepository
        _repository;

    public GiftProcessor(
        IGiftRepository repository)
    {
        _repository = repository;
    }

    public void Process(
        Gift gift)
    {
        // implementation
    }
}
```

The private repository field is:

```EXAMPLE / FLOW
Encapsulation
```

The interface is:

```EXAMPLE / FLOW
Abstraction
```

Two Major C# Abstraction Mechanisms

In object-oriented C#, the two major mechanisms are:

```EXAMPLE / FLOW
Abstract Classes

Interfaces
```

Both can define a contract.

But they solve different design problems.

Abstract Classes

Definition

An **abstract class is a class intended to act as a base abstraction and which cannot be instantiated directly**.

Example:

```
C#
public abstract class FinancialTransaction
{
}
```

This is invalid:

```
C#
// new FinancialTransaction();
```

Instead, another class derives from it:

```
C#
public sealed class GiftTransaction :
    FinancialTransaction
{
}
```

An abstract class can contain both implemented members and abstract members. It doesn't actually need to contain an abstract member to be declared abstract.

Why Use an Abstract Class?

Use an abstract class when several related classes need:

```
EXAMPLE / FLOW
A shared identity/type relationship

Shared implementation

Shared state

Constructors

Protected extension points

Required specialized behavior
```

Example hierarchy:

```
EXAMPLE / FLOW
FinancialTransaction
        ↓
  ┌─────────────┐
  ↓             ↓
GiftTransaction RefundTransaction
```

These objects are all genuinely:

EXAMPLE / FLOW

```
FinancialTransactions
```

Abstract Class Example

C#

```
public abstract class FinancialTransaction
{
    protected FinancialTransaction(
        long transactionId,
        decimal amount)
    {
        if (transactionId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(transactionId));
        }

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        TransactionId = transactionId;

        Amount = amount;
    }

    public long TransactionId
    {
        get;
    }

    public decimal Amount
    {
        get;
    }

    public string GetBaseSummary()
    {
        return
            $"{TransactionId}: {Amount:C2}";
    }
}
```

The class is abstract even though:

EXAMPLE / FLOW

```
GetBaseSummary()
```

already has an implementation.

Abstract Classes Can Have State

Unlike the traditional interface contract model, an abstract class can naturally own instance fields and state.

Example:

```C#
public abstract class FinancialTransaction
{
    private readonly long _tenantId;

    protected FinancialTransaction(
        long tenantId)
    {
        _tenantId = tenantId;
    }

    public long TenantId
        =>
        _tenantId;
}
```

Derived objects include this base state.

Abstract Classes Can Have Constructors

Even though you cannot write:

```C#
new FinancialTransaction();
```

an abstract class can have constructors.

Why?

Derived constructors use them.

Example:

```C#
public abstract class FinancialTransaction
{
    protected FinancialTransaction(
        long tenantId,
        decimal amount)
    {
        TenantId = tenantId;

        Amount = amount;
    }

    public long TenantId
    {
        get;
    }

    public decimal Amount
    {
        get;
    }
}
```

Derived:

```C#
public sealed class GiftTransaction :
    FinancialTransaction
{
```

```
    public GiftTransaction(
        long tenantId,
        decimal amount)
        : base(
            tenantId,
            amount)
    {
    }
}
```

ABSTRACTION / INTERFACES

Why Abstract Constructors Are Usually Protected

The class cannot be instantiated directly.

So its constructor is often:

C#
```
protected
```

because its main consumers are:

EXAMPLE / FLOW
```
Derived classes
```

Example:

C#
```
protected FinancialTransaction(...)
```

A Constructor Itself Is Not abstract

This is not valid:

C#
```
// public abstract FinancialTransaction();
```

Constructors aren't inherited/overridden like virtual methods.

The abstract class defines actual constructors that derived constructors can call.

Abstract Members

Definition

An abstract member declares:

> Every concrete derived class must provide an implementation.

Example:

C#
```
public abstract class FinancialTransaction
{
    public abstract string
        GetSummary();
}
```

There is no method body.

Abstract Method Syntax

```C#
public abstract string GetSummary();
```

Notice:

```EXAMPLE / FLOW
abstract

No method body

Ends with ;
```

A class containing an abstract member must itself be abstract.

Derived Implementation

```C#
public sealed class GiftTransaction :
    FinancialTransaction
{
    public override string GetSummary()
    {
        return "Gift";
    }
}
```

The derived implementation uses:

```EXAMPLE / FLOW
override
```

because abstract members participate in virtual dispatch.

Abstract Members Are Polymorphic

Base:

```C#
public abstract string GetSummary();
```

Derived:

```C#
public override string GetSummary()
```

Call:

```C#
FinancialTransaction transaction = new GiftTransaction();

transaction.GetSummary();
```

Runtime executes:

EXAMPLE / FLOW

```
GiftTransaction.GetSummary()
```

ABSTRACTION / INTERFACES

This connects Topic 15 directly to Topic 14.

Abstract Class May Mix Abstract and Concrete Members

Example:

C#

```
public abstract class FinancialTransaction
{
    public decimal Amount
    {
        get;
        protected set;
    }

    public bool HasPositiveAmount()
    {
        return Amount > 0m;
    }

    public abstract string
        GetSummary();
}
```

Concrete shared behavior:

EXAMPLE / FLOW

```
HasPositiveAmount()
```

Required specialized behavior:

EXAMPLE / FLOW

```
GetSummary()
```

Why This Is Useful

An abstract base class can say:

EXAMPLE / FLOW

```
Every FinancialTransaction
shares these things
```

while also saying:

EXAMPLE / FLOW

```
Every concrete transaction
must define these things differently
```

Abstract Property

Properties can be abstract too.

Example:

```C#
public abstract class FinancialTransaction
{
    public abstract string
        TransactionType
    {
        get;
    }
}
```

Derived:

```C#
public sealed class GiftTransaction :
    FinancialTransaction
{
    public override string
        TransactionType
        =>
        "Gift";
}
```

Abstract properties behave similarly to abstract methods: the base defines the property contract, while the derived class supplies an override.

Abstract Get and Set

You can require accessors.

Example:

```C#
public abstract class Entity
{
    public abstract string Name
    {
        get;
        protected set;
    }
}
```

A derived class must implement a compatible property contract.

Abstract Class Can Implement an Interface

Example:

```C#
public interface IValidatable
{
    void Validate();
}
```

Abstract base:

```C#
public abstract class FinancialTransaction :
    IValidatable
{
```

```
    public abstract void Validate();
}
```


Concrete derived class:

```C#
public sealed class GiftTransaction :
    FinancialTransaction
{
    public override void Validate()
    {
        // Gift validation.
    }
}
```

An abstract class may map an interface requirement to an abstract member so that concrete derived classes complete the implementation.

Abstract vs Concrete Class

Abstract:

```C#
public abstract class Transaction
{
}
```

Cannot be directly instantiated.

Concrete:

```C#
public sealed class Gift :
    Transaction
{
}
```

Can be instantiated if its construction requirements are satisfied.

When an Abstract Class Is Appropriate

Consider an abstract class when all of these are broadly true:

```EXAMPLE / FLOW
Types are closely related

A real IS-A relationship exists

Common state exists

Common implementation exists

Derived types require shared lifecycle behavior

Some behavior must be specialized
```

When Abstract Class May Be Wrong

If types only share a capability:

EXAMPLE / FLOW

```
Can log

Can export

Can validate

Can notify
```

but aren't naturally members of one class hierarchy, an interface is usually better.

Interfaces

Definition

An **interface defines a contract that implementing types agree to satisfy**.

Example:

C#

```
public interface IGiftRepository
{
    Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Any class, record, or struct implementing the interface must satisfy its required contract members. Interfaces can also inherit multiple other interfaces.

Interface Mental Model

EXAMPLE / FLOW

```
Interface
    →
Contract

Implementation
    →
How contract is fulfilled
```

Example:

EXAMPLE / FLOW

```
IGiftRepository
        ↓
  ┌─────┴──────────┐
  ↓                ↓
SqlGiftRepository   FakeGiftRepository
```

Interface Naming Convention

In .NET, interfaces conventionally begin with:

EXAMPLE / FLOW

```
I
```

Examples:

EXAMPLE / FLOW

```
IDisposable

IEnumerable<T>

IComparable<T>

ILogger

IGiftRepository

IPaymentProcessor
```

For your project:

EXAMPLE / FLOW

```
IGiftService

IGiftRepository

IGiftReceiptSender
```

are clear interface names.

Basic Interface Syntax

C#

```
public interface IGiftRepository
{
}
```

Implementation:

C#

```
public sealed class SqlGiftRepository :
    IGiftRepository
{
}
```

The colon means:

EXAMPLE / FLOW

```
implements
```

when the listed type is an interface.

A Type Can Implement Multiple Interfaces

Example:

C#

```
public sealed class GiftService :
    IGiftReader,
    IGiftWriter,
    IAuditable
{
}
```

C# classes have one direct base class but can implement multiple interfaces. Interfaces themselves can inherit multiple interfaces.

Class + Interfaces

Example:

C#
```
public class GiftService :
    BaseService,
    IGiftReader,
    IGiftWriter
{
}
```

When both a base class and interfaces are listed:

EXAMPLE / FLOW
```
Base class first
then interfaces
```

is required.

Interface Method

Traditional contract:

C#
```
public interface IGiftRepository
{
    Task<Gift?> GetAsync(
        long giftId,
        CancellationToken cancellationToken);
}
```

No implementation body is supplied.

The implementing class must provide it.

Implementation

C#
```
public sealed class SqlGiftRepository :
    IGiftRepository
{
    public Task<Gift?> GetAsync(
        long giftId,
        CancellationToken cancellationToken)
    {
        // Database implementation.

        throw new NotImplementedException();
    }
}
```

Why public?

A normal implicit implementation of a public interface member must be publicly accessible.

This won't satisfy the normal public contract:

```C#
// private Task<Gift?> GetAsync(...)
```

If you don't want the member exposed directly on the class surface, use explicit interface implementation instead.

Interface Properties

Interfaces can define property contracts.

Example:

```C#
public interface ITenantScoped
{
    long TenantId
    {
        get;
    }
}
```

Implementation:

```C#
public sealed class Gift :
    ITenantScoped
{
    public long TenantId
    {
        get;
    }

    public Gift(
        long tenantId)
    {
        TenantId = tenantId;
    }
}
```

Interface Property Does Not Create Storage

This:

```C#
public interface ITenantScoped
{
    long TenantId
    {
        get;
    }
}
```

does not create an automatic backing field the way an auto-property in a class does.

It says:

EXAMPLE / FLOW

```
Implementing type must provide
this property contract
```

Microsoft explicitly distinguishes interface property declarations from class auto-properties.

Interface with get/set

C#

```
public interface INamed
{
    string Name
    {
        get;
        set;
    }
}
```

Implementer must provide compatible access.

Prefer Narrow Contracts

If callers only need:

EXAMPLE / FLOW

```
get
```

don't unnecessarily demand:

EXAMPLE / FLOW

```
set
```

Example:

C#

```
public interface IGiftSummary
{
    decimal TotalAmount
    {
        get;
    }
}
```

This contract is narrower than:

C#

```
decimal TotalAmount
{
    get;
    set;
}
```

Interface Events

Interfaces can declare events.

Example:

```C#
public interface IGiftProcessor
{
    event EventHandler?
        GiftProcessed;
}
```

The implementation provides the actual event behavior.

Events are Topic 23, so this is only an awareness example.

Interface Indexers

Interfaces can also declare indexers.

Example:

```C#
public interface IGiftLookup
{
    Gift? this[long giftId]
    {
        get;
    }
}
```

Implementation supplies the indexer behavior.

Traditional Interface Rule

Historically, developers often summarized interfaces as:

EXAMPLE / FLOW

```
Interface contains declarations only
```

That used to be largely correct.

Modern C# is more capable.

Interfaces can now include implementations and various static members.

So the modern mental model should be:

> **An interface defines a contract, and modern interfaces may optionally provide some implementation associated with that contract.**

Interface Cannot Be Instantiated Directly

Invalid:

```C#
// IGiftRepository repository = //     new IGiftRepository();
```

You need an implementing object:

```C#
IGiftRepository repository = new SqlGiftRepository();
```

Interface Reference

The variable:

```
C#
IGiftRepository repository
```

contains a reference to an object implementing:

```
EXAMPLE / FLOW
IGiftRepository
```

The runtime object may be:

```
EXAMPLE / FLOW
SqlGiftRepository

FakeGiftRepository

CachedGiftRepository
```

This provides interface polymorphism.

Interface Polymorphism

```
C#
IGiftRepository repository = new SqlGiftRepository();
```

Later:

```
C#
IGiftRepository repository = new FakeGiftRepository();
```

Consumer code still calls:

```
C#
await repository.SaveAsync(...);
```

Same contract.

Different implementation.

Implicit Interface Implementation

The most common implementation style is:

```
C#
public interface IReceiptSender
{
    Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Implementation:

```C#
public sealed class EmailReceiptSender :
    IReceiptSender
{
    public Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        // Email implementation.

        return Task.CompletedTask;
    }
}
```

The method is directly visible:

```C#
EmailReceiptSender sender = new();

sender.SendAsync(...);
```

and through the interface:

```C#
IReceiptSender sender = new EmailReceiptSender();

sender.SendAsync(...);
```

Explicit Interface Implementation

Sometimes you don't want the interface member to appear as an ordinary public member of the class.

C# provides:

EXAMPLE / FLOW

```
Explicit Interface Implementation
```

Syntax:

```C#
public interface IAuditable
{
    void WriteAudit();
}
```

Implementation:

```C#
public sealed class Gift :
    IAuditable
{
    void IAuditable.WriteAudit()
    {
        Console.WriteLine(
            "Gift audit.");
    }
}
```

Notice:

EXAMPLE / FLOW

```
No public modifier
```

and member name includes:

EXAMPLE / FLOW

```
IAuditable.
```

Calling Explicit Implementation

This does not work:

C#

```
Gift gift = new Gift();

// gift.WriteAudit();
```

Instead:

C#

```
IAuditable auditable = gift;

auditable.WriteAudit();
```

An explicitly implemented instance member is available through the interface reference rather than as an ordinary class member.

Why Explicit Interface Implementation Exists

Major uses:

EXAMPLE / FLOW

```
Hide interface-specific members
from normal class surface

Resolve member-name conflicts

Provide different implementations
for two interfaces

Expose behavior only through
the interface abstraction
```

Two Interfaces, Same Method

Suppose:

C#

```
public interface IInternalNotifier
{
    void Notify();
}

public interface IExternalNotifier
{
    void Notify();
}
```

One implementation can satisfy both:

```C#
public sealed class GiftNotifier :
    IInternalNotifier,
    IExternalNotifier
{
    public void Notify()
    {
        Console.WriteLine(
            "Notification.");
    }
}
```

All three calls use the same implementation:

```C#
GiftNotifier notifier = new();

notifier.Notify();

((IInternalNotifier)notifier)
    .Notify();

((IExternalNotifier)notifier)
    .Notify();
```

One public implementation can satisfy identical interface contracts.

Different Behavior for Each Interface

Suppose the two contracts require different semantics.

```C#
public sealed class GiftNotifier :
    IInternalNotifier,
    IExternalNotifier
{
    void IInternalNotifier.Notify()
    {
        Console.WriteLine(
            "Internal notification.");
    }

    void IExternalNotifier.Notify()
    {
        Console.WriteLine(
            "External notification.");
    }
}
```

Then:

```C#
IInternalNotifier internalNotifier = new GiftNotifier();

IExternalNotifier externalNotifier = new GiftNotifier();
```

Calls:

```
C#
internalNotifier.Notify();
```

and:

```
C#
externalNotifier.Notify();
```

execute different implementations.

This is one of the primary uses of explicit interface implementation.

Explicit Interface Implementations Have No Access Modifier

This is invalid:

```
C#
// public void IAuditable.WriteAudit()
```

An explicit implementation doesn't declare an ordinary access modifier.

Its availability is controlled through the interface reference.

When Explicit Implementation Is Useful

Use it when:

```
EXAMPLE / FLOW
Two interfaces collide

Interface terminology doesn't fit
the class's normal public API

Member should only be used
through abstraction

Two interface contracts need
different implementations
```

When Not to Use Explicit Implementation

Don't hide a normal useful operation merely to make the code look sophisticated.

If callers naturally need:

```
C#
repository.SaveAsync(...)
```

directly on the class, normal public implementation is simpler.

Interface Inheritance

An interface can inherit another interface.

Example:

```
C#
public interface IGiftReader
{
```

```
    Task<Gift?> GetAsync(
        long giftId,
        CancellationToken cancellationToken);
}
```



```C#
public interface IGiftWriter
{
    Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Combined:

```C#
public interface IGiftRepository :
    IGiftReader,
    IGiftWriter
{
}
```

Interfaces can inherit multiple base interfaces. A type implementing the derived interface must satisfy inherited members as well.

Implementation

```C#
public sealed class SqlGiftRepository :
    IGiftRepository
{
    public Task<Gift?> GetAsync(
        long giftId,
        CancellationToken cancellationToken)
    {
        throw new NotImplementedException();
    }

    public Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        throw new NotImplementedException();
    }
}
```

Because:

```
EXAMPLE / FLOW
IGiftRepository
    →
IGiftReader
+
IGiftWriter
```

the implementation satisfies all inherited contracts.

Interface Inheritance Flow

EXAMPLE / FLOW

```
IGiftReader      IGiftWriter
      \           /
       \         /
        ↓       ↓
      IGiftRepository
            ↓
     SqlGiftRepository
```

Interface Inheritance Is Different from Class Inheritance

Class inheritance may include:

EXAMPLE / FLOW

```
Instance state

Constructors

Protected implementation

Shared concrete behavior
```

Interface inheritance mainly combines/refines:

EXAMPLE / FLOW

```
Contracts / capabilities
```

although modern interfaces can also provide default member implementations.

Capability-Oriented Interfaces

A useful way to think about many interfaces is:

EXAMPLE / FLOW

```
Can this object do X?
```

Examples:

EXAMPLE / FLOW

```
IEnumerable
    →
Can be enumerated

IDisposable
    →
Can be disposed

IComparable<T>
    →
Can be compared
```

Your application:

EXAMPLE / FLOW

```
IGiftReader
    →
```

```
Can read gifts
*
IGiftWriter
    →
Can save gifts

IReceiptSender
    →
Can send receipts
```

Abstract Class vs Interface

This is one of the most important interview and design questions.

Abstract Class	Interface
Represents a base class	Represents a contract/capability
Can contain instance state	No ordinary instance fields
Can have constructors	No ordinary instance constructors
Can contain concrete methods	Can provide default methods in modern C#
Can contain abstract methods	Can require abstract members
Can use protected/private instance members	Modern interfaces support member accessibility with rules
Class can inherit only one	Type can implement multiple
Best for closely related types	Best for possibly unrelated implementations
Supports shared implementation naturally	Usually focuses on behavioral contract
Participates in class hierarchy	Can cross class hierarchies
Can have normal instance fields	Interface can have static fields, not instance state fields

Modern interfaces are more powerful than historical C# interfaces, but the fundamental design distinction remains.

Simple Decision Rule

Ask:

EXAMPLE / FLOW

```
Do these types share
identity/state/implementation
because they truly belong
to one inheritance hierarchy?
```

If yes:

EXAMPLE / FLOW

```
Abstract class may fit.
```

Ask:

EXAMPLE / FLOW

```
Do these types merely share
a capability/contract?
```

If yes:

EXAMPLE / FLOW

```
Interface may fit.
```

Example — Abstract Class Fits

EXAMPLE / FLOW

```
FinancialTransaction
        ↓
GiftTransaction

RefundTransaction

AdjustmentTransaction
```

All share:

EXAMPLE / FLOW

```
TransactionId

TenantId

Amount

CreatedAt
```

and perhaps common behavior.

An abstract base class can make sense.

Example — Interface Fits

Consider:

EXAMPLE / FLOW

```
EmailReceiptSender

WebhookReceiptSender

PrintReceiptSender
```

They don't necessarily share a meaningful base-class identity.

But all can:

EXAMPLE / FLOW

```
SendReceipt
```

So:

C#

```
public interface IReceiptSender
{
    Task SendAsync(...);
}
```

fits well.

Combining Abstract Class and Interfaces

They aren't mutually exclusive.

Example:

```C#
public abstract class FinancialTransaction :
    ITenantScoped,
    IAuditable
{
}
```

Derived classes inherit:

```EXAMPLE / FLOW
FinancialTransaction
```

and also participate in:

```EXAMPLE / FLOW
ITenantScoped

IAuditable
```

Example

```C#
public interface ITenantScoped
{
    long TenantId
    {
        get;
    }
}
```

```C#
public abstract class FinancialTransaction :
    ITenantScoped
{
    protected FinancialTransaction(
        long tenantId)
    {
        TenantId = tenantId;
    }

    public long TenantId
    {
        get;
    }
}
```

The abstract class provides the common implementation of the interface contract.

Default Interface Implementations

Modern C# interfaces may provide a method body.

Example:

```C#
public interface IReferenceFormatter
{
    string Prefix
    {
        get;
    }

    public string FormatReference(
        long id)
    {
        return
            $"{Prefix}-{id}";
    }
}
```

The `FormatReference` method provides a default implementation.

Default interface implementations were introduced primarily to allow interfaces to evolve by adding behavior without requiring every existing implementing type to immediately provide that implementation.

Implementing Class

```C#
public sealed class GiftReferenceFormatter :
    IReferenceFormatter
{
    public string Prefix
        =>
        "GFT";
}
```

It doesn't explicitly implement:

EXAMPLE / FLOW

```
FormatReference
```

so interface dispatch can use the default implementation.

Important Default Interface Rule

This:

```C#
GiftReferenceFormatter formatter = new();
```

does not necessarily let you write:

```C#
// formatter.FormatReference(1001);
```

when the class itself hasn't declared that method.

Instead:

```C#
IReferenceFormatter formatter = new GiftReferenceFormatter();
```

```
string value = formatter.FormatReference(
    1001);
```


Default interface members are available through the interface abstraction rather than being injected into the implementing class's ordinary public surface.

Why This Matters

Default interface implementation does **not** mean:

```
EXAMPLE / FLOW
Interface method becomes
an inherited class method
```

It means:

```
EXAMPLE / FLOW
Interface dispatch has
a default implementation available
```

Class Can Provide Its Own Implementation

```
C#
public sealed class GiftReferenceFormatter :
    IReferenceFormatter
{
    public string Prefix
        =>
        "GFT";

    public string FormatReference(
        long id)
    {
        return
            $"GIFT-{id:D10}";
    }
}
```

Now calls through:

```
EXAMPLE / FLOW
IReferenceFormatter
```

use the class implementation.

No override Keyword Required

Notice:

```
C#
public string FormatReference(...)
```

not:

```
C#
// public override string FormatReference(...)
```

A class implementing an interface member doesn't use class-inheritance `override` syntax simply because the interface had a default implementation.

Interface implementation and class overriding have different mechanisms.

Default Interface Methods Are Not Base-Class Replacement

Avoid thinking:

EXAMPLE / FLOW

```
Interfaces have methods now
therefore abstract classes are obsolete.
```

They are not.

Abstract classes still naturally provide:

EXAMPLE / FLOW

```
Instance state

Constructors

Protected state/operations

Shared object lifecycle

Class-hierarchy semantics
```

Default interface members primarily help interface contract evolution and reusable contract-level behavior.

Do Not Overuse Default Implementations

If an interface becomes:

EXAMPLE / FLOW

```
50 methods

20 helper functions

large algorithm implementation

complex mutable behavior
```

you may be building a disguised base class.

Ask whether:

EXAMPLE / FLOW

```
Abstract class

Service

Composition
```

would be clearer.

Modern Interface Accessibility Rules

Traditional interface declarations like:

```
C#
public interface IGiftReader
{
    Gift? Get(
        long id);
}
```

have abstract members that are implicitly:

```
EXAMPLE / FLOW
public
```

An abstract interface member without an implementation can't use a different accessibility.

Modern interface members that **have an implementation** can use explicit accessibility such as:

```
EXAMPLE / FLOW
public

private

protected

internal
```

and if an implemented member has no access modifier, its default accessibility is `private`.

Practical Recommendation

For ordinary application contracts, keep interfaces simple and obvious:

```
C#
public interface IGiftRepository
{
    Task<Gift?> GetAsync(...);

    Task SaveAsync(...);
}
```

Don't introduce advanced interface accessibility unless it genuinely improves the design.

Private Interface Helpers

Modern interfaces can use private implementation helpers for default behavior.

Conceptually:

```
C#
public interface IReferenceFormatter
{
    public string Format(
        long id)
    {
        return Normalize(
            $"GFT-{id}");
    }

    private static string Normalize(
        string value)
    {
        return value
```

```
            .Trim()
            .ToUpperInvariant();
    }
}
```


The private helper supports interface implementation logic without becoming part of the public interface contract.

Static Interface Members

Modern interfaces can contain various static members, including:

EXAMPLE / FLOW

```
Static methods

Static properties

Static fields

Constants

Operators

Static constructors
```

They can also declare advanced:

EXAMPLE / FLOW

```
static abstract

static virtual
```

members.

Static Abstract Interface Members

This is an advanced modern C# feature.

An interface can require that an implementing type provide a static member.

Example:

C#

```
public interface IEntityFactory<TSelf>
    where TSelf :
        IEntityFactory<TSelf>
{
    static abstract TSelf Create(
        long id);
}
```

Implementation:

C#

```
public sealed class GiftReference :
    IEntityFactory<GiftReference>
{
    private GiftReference(
        long id)
    {
        Id = id;
```

```
    }

    public long Id
    {
        get;
    }

    public static GiftReference Create(
        long id)
    {
        if (id <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(id));
        }

        return new GiftReference(
            id);
    }
}
```

The implementing member must itself be static.

Generic Use

C#

```
static T CreateEntity<T>(
    long id)
    where T :
        IEntityFactory<T>
{
    return T.Create(
        id);
}
```

Call:

C#

```
GiftReference reference = CreateEntity<GiftReference>(
        1001);
```

This enables generic algorithms to work with required static behavior.

Why Static Abstract Interface Members Exist

Historically, interfaces primarily described:

EXAMPLE / FLOW

```
Instance behavior
```

But generic algorithms sometimes need:

EXAMPLE / FLOW

```
Static operations

Operators

Factory-like behavior

Number-like behavior
```

Static abstract interface members enable that kind of type-level generic contract and are used extensively by modern generic-math abstractions.

Should Normal Business Interfaces Use static abstract?

Usually not.

For interfaces such as:

EXAMPLE / FLOW

```
IGiftRepository

IGiftService

IReceiptSender
```

ordinary instance members are normally the right design.

Use `static abstract` when generic algorithms genuinely require type-level behavior.

static virtual

Interfaces may also declare:

EXAMPLE / FLOW

```
static virtual
```

members.

This lets an interface provide a static default that implementing types can specialize.

This is an advanced generic-programming feature rather than something most ASP.NET Core business services need daily.

Interface Segregation Preview

Suppose you define:

C#

```
public interface IGiftRepository
{
    Task<Gift?> GetAsync(...);

    Task SaveAsync(...);

    Task DeleteAsync(...);

    Task ExportAsync(...);

    Task EmailAsync(...);

    Task GeneratePdfAsync(...);

    Task ReconcilePaymentAsync(...);
}
```

Every implementation now depends on a huge contract.

That can be difficult.

Sometimes smaller interfaces are better:

C#
```
public interface IGiftReader
{
    Task<Gift?> GetAsync(...);
}
```

C#
```
public interface IGiftWriter
{
    Task SaveAsync(...);
}
```

This idea becomes the:

EXAMPLE / FLOW
```
Interface Segregation Principle
```

in SOLID Topic 31.

Interface Should Represent a Cohesive Contract

Good:

EXAMPLE / FLOW
```
IGiftRepository
    →
Persistence operations
```

Bad:

EXAMPLE / FLOW
```
IEverything
    →
Database
Email
Logging
Payments
PDF
Authorization
Reports
```

An interface should express a coherent responsibility.

Do Not Create Interface Per Class Automatically

A common over-engineering pattern is:

EXAMPLE / FLOW
```
GiftCalculator
IGiftCalculator

CurrencyFormatter
ICurrencyFormatter

StringNormalizer
IStringNormalizer
```

```
GiftMath
IGiftMath
```

even when:

EXAMPLE / FLOW

```
Only one implementation exists

No substitution is useful

No boundary exists

Testing doesn't benefit

Behavior is simple and pure
```

Interfaces are valuable when they create a meaningful abstraction.

They aren't mandatory companions for every class.

Good Reasons for an Interface

Common reasons include:

EXAMPLE / FLOW

```
Multiple implementations

External-resource boundary

Dependency inversion

Testing substitution

Cross-cutting capability

Plugin-style architecture

Different infrastructure providers
```

Repository Boundary Example

Application layer:

EXAMPLE / FLOW

```
IGiftRepository
```

Infrastructure:

EXAMPLE / FLOW

```
SqlGiftRepository
```

Test:

EXAMPLE / FLOW

```
FakeGiftRepository
```

This is a meaningful abstraction boundary.

HTTP Boundary Example

C#

```
public interface IPaymentGateway
{
    Task<PaymentResult>
        ChargeAsync(
            PaymentRequest request,
            CancellationToken cancellationToken);
}
```

Implementations:

EXAMPLE / FLOW

```
StripePaymentGateway

MockPaymentGateway

SandboxPaymentGateway
```

The business service shouldn't need to understand HTTP mechanics.

Abstraction in ASP.NET Core

ASP.NET Core heavily uses abstractions.

Common examples include:

EXAMPLE / FLOW

```
ILogger<T>

IConfiguration

IHostEnvironment

IServiceProvider
```

and application-specific interfaces such as:

EXAMPLE / FLOW

```
IGiftService

IGiftRepository

IPaymentProcessor
```

Dependency injection then supplies implementations.

Dependency Injection Flow

EXAMPLE / FLOW

```
Controller
    →
IGiftService
    →
GiftService
    →
IGiftRepository
```

```
    →
SqlGiftRepository
```

Each layer can depend primarily on the contract it needs.

Example Service Contract

```C#
public interface IGiftService
{
    Task<long> CreateAsync(
        CreateGiftRequest request,
        CancellationToken cancellationToken);
}
```

Implementation:

```C#
public sealed class GiftService :
    IGiftService
{
    private readonly IGiftRepository
        _repository;

    public GiftService(
        IGiftRepository repository)
    {
        _repository = repository;
    }

    public async Task<long> CreateAsync(
        CreateGiftRequest request,
        CancellationToken cancellationToken)
    {
        Gift gift = new Gift(
                request.ConstituentId,
                request.Amount);

        await _repository.SaveAsync(
            gift,
            cancellationToken);

        return gift.GiftId;
    }
}
```

Controller Depends on Abstraction

```C#
public sealed class GiftsController :
    ControllerBase
{
    private readonly IGiftService
        _giftService;

    public GiftsController(
        IGiftService giftService)
    {
        _giftService = giftService;
```

```
    }
}
```


Controller doesn't need to know:

```
EXAMPLE / FLOW
Repository type

DbContext

SQL tables

Payment implementation
```

It knows:

```
EXAMPLE / FLOW
IGiftService
```

Abstraction Boundary

```
EXAMPLE / FLOW
GiftsController
        ↓
IGiftService
        ↓
GiftService
        ↓
IGiftRepository
        ↓
SqlGiftRepository
        ↓
Database
```

Each abstraction hides lower-level details.

Testing Benefit

Production:

```
C#
IGiftRepository repository = new SqlGiftRepository(...);
```

Testing:

```
C#
IGiftRepository repository = new FakeGiftRepository();
```

`GiftService` doesn't change.

That is a major practical benefit of interface abstraction.

Test Example

```
C#
public sealed class FakeGiftRepository :
    IGiftRepository
{
```

```
    public List<Gift> SavedGifts
    {
        get;
    } = [];

    public Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        SavedGifts.Add(
            gift);

        return Task.CompletedTask;
    }
}
```

Test:

```C#
var repository = new FakeGiftRepository();

var service = new GiftService(
        repository);
```

No database required.

Abstract Class + Interface Together

A realistic design may use both.

```C#
public interface IAuditable
{
    string GetAuditDescription();
}
```

Base:

```C#
public abstract class FinancialTransaction :
    IAuditable
{
    protected FinancialTransaction(
        long transactionId,
        decimal amount)
    {
        TransactionId = transactionId;

        Amount = amount;
    }

    public long TransactionId
    {
        get;
    }

    public decimal Amount
    {
        get;
    }

    public abstract string
```

```
        GetAuditDescription();
}
```


Derived:

```C#
public sealed class GiftTransaction :
    FinancialTransaction
{
    public GiftTransaction(
        long transactionId,
        decimal amount,
        long constituentId)
        : base(
            transactionId,
            amount)
    {
        ConstituentId = constituentId;
    }

    public long ConstituentId
    {
        get;
    }

    public override string
        GetAuditDescription()
    {
        return
            $"Gift {TransactionId} "
            +
            $"for constituent {ConstituentId}.";
    }
}
```

Here:

```
EXAMPLE / FLOW
FinancialTransaction
```

models shared inheritance.

```
EXAMPLE / FLOW
IAuditable
```

models a capability contract.

Why Not Put Everything in an Interface?

Suppose transaction types all need:

```
EXAMPLE / FLOW
TransactionId

TenantId

Amount

CreatedAtUtc
```

plus validation and constructor logic.

An interface can describe properties:

C#
```
public interface ITransaction
{
    long TransactionId
    {
        get;
    }

    decimal Amount
    {
        get;
    }
}
```

but it doesn't naturally give each object the same protected instance state and constructor lifecycle.

An abstract class may be more appropriate when shared state/implementation is fundamental.

Why Not Put Everything in an Abstract Class?

Suppose unrelated types need:

EXAMPLE / FLOW
```
IAuditable
```

Examples:

EXAMPLE / FLOW
```
Gift

Campaign

AppUser

Task

Interaction
```

It would be wrong to force all of them into one class hierarchy solely to gain auditing capability.

Use:

EXAMPLE / FLOW
```
IAuditable
```

instead.

Multiple Capabilities

A `Gift` might implement:

C#
```
public sealed class Gift :
    ITenantScoped,
    IAuditable,
    IValidatable
{
}
```

This says:

EXAMPLE / FLOW

```
Gift is tenant-scoped

Gift can be audited

Gift can be validated
```

without requiring three base classes.

Class Inheritance vs Interface Capabilities

EXAMPLE / FLOW

```
Class hierarchy
    →
What kind of thing is this?

Interface
    →
What contract/capability does this support?
```

This is a useful design heuristic.

Default Interface Method Versioning

Historically:

C#

```
public interface INotifier
{
    void Notify();
}
```

If a library later added:

C#

```
void NotifyUrgently();
```

every existing implementation would need to implement the new member.

A default implementation can provide a compatible starting behavior:

C#

```
public interface INotifier
{
    void Notify();

    public void NotifyUrgently()
    {
        Notify();
    }
}
```

Existing implementations can still rely on the default while new implementations may provide specialized behavior. This contract-evolution scenario is a primary reason default interface methods were introduced.

Default Methods Still Require Care

Don't assume:

EXAMPLE / FLOW

```
Adding a default member
=
zero versioning concerns forever
```

Public API evolution still requires careful compatibility design.

Default interface methods simply provide another tool for evolving contracts.

Interface Members and State

Traditional instance interfaces should be viewed as contracts, not containers of per-object fields.

Modern interfaces may declare static state, but they don't provide ordinary instance fields for each implementing object.

If you require substantial shared instance state:

EXAMPLE / FLOW

```
Abstract class
```

is usually more natural.

Interface Cannot Require Constructor Syntax

An ordinary interface cannot say:

EXAMPLE / FLOW

```
Every implementation must have:

new T(int id)
```

through normal instance interface syntax.

Modern generic scenarios may express factory-like static contracts using:

EXAMPLE / FLOW

```
static abstract
```

but that's different from ordinary instance construction.

Abstract Class Constructors Can Enforce Shared State

Example:

C#

```
public abstract class TenantEntity
{
    protected TenantEntity(
        long tenantId)
    {
        if (tenantId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(tenantId));
        }
```

```
        TenantId = tenantId;
    }

    public long TenantId
    {
        get;
    }
}
```

Every derived class must route through valid base construction.

That is something an ordinary instance interface doesn't provide.

Common Mistake — Abstraction = Interface

Abstraction is a broader concept.

You can create abstraction using:

EXAMPLE / FLOW

```
Interfaces

Abstract classes

Public APIs

Methods

Services

Modules

Encapsulation
```

An interface is only one mechanism.

Common Mistake — Interface = No Implementation

Historically common.

Modern C# allows:

EXAMPLE / FLOW

```
Default interface members

Static members

Static abstract members
```

so the statement is no longer universally correct.

Common Mistake — Interface Has Auto-Properties

This:

C#

```
public interface IGift
{
    decimal Amount
    {
        get;
```

```
        set;
    }
}
```


doesn't allocate hidden instance storage.

It specifies a property contract that implementations must satisfy.

Common Mistake — Abstract Class Must Have Abstract Members

No.

This is legal:

```C#
public abstract class BaseService
{
    public void Log()
    {
    }
}
```

An abstract class may exist simply to prevent direct construction while providing shared base behavior.

Common Mistake — Abstract Class Can Be Instantiated

Invalid:

```C#
// var value = //     new FinancialTransaction();
```

An abstract class is incomplete as a directly constructible type.

Common Mistake — Abstract Method Has Body

Invalid concept:

```C#
// public abstract void Process()
// {
// }
```

An abstract class method declaration does not provide its own ordinary body.

If you want default behavior plus optional specialization:

```C#
public virtual void Process()
{
}
```

may be appropriate.

Common Mistake — Interface Can Solve Multiple Class Inheritance

Interfaces let a type implement multiple contracts.

They do **not** mean the class receives multiple sets of shared base-class state.

For example:

```C#
class Gift :
    IAuditable,
    IValidatable
```

implements two contracts.

It isn't the same as inheriting full implementation/state from two classes.

Common Mistake — Interface Everywhere

Do not create abstractions without a meaningful boundary.

Bad reasoning:

```EXAMPLE / FLOW
Every class needs interface
because DI exists.
```

Better:

```EXAMPLE / FLOW
Does this dependency represent
a useful replaceable contract?
```

Common Mistake — Fat Interfaces

Avoid:

```C#
public interface IGiftEverything
{
    void Save();

    void Delete();

    void Email();

    void Print();

    void Export();

    void Authorize();

    void Charge();

    void Refund();

    void Audit();
}
```

Too many responsibilities create coupling.

Common Mistake — Leaking Implementation Details

Bad interface:

```C#
public interface IGiftRepository
{
    SqlConnection GetConnection();

    DbCommand BuildInsertCommand();

    Task SaveAsync(
        Gift gift);
}
```

Now higher layers understand persistence internals.

Better abstraction:

```C#
Task SaveAsync(
    Gift gift,
    CancellationToken cancellationToken);
```

Expose what consumers need.

Common Mistake — Repository Returning EF-Specific Internals

If application code must know:

```EXAMPLE / FLOW
DbContext

EntityEntry

SQL connection details
```

the repository abstraction may no longer hide infrastructure meaningfully.

Design the contract around application needs.

Common Mistake — Interface Named After Implementation

Weak:

```EXAMPLE / FLOW
ISqlGiftRepository
```

if the purpose is to hide SQL implementation details.

Usually better:

```EXAMPLE / FLOW
IGiftRepository
```

Implementation:

```EXAMPLE / FLOW
SqlGiftRepository
```

The abstraction describes responsibility rather than implementation technology.

Common Mistake — Explicit Implementation Without Reason

This:

```
C#
Task IGiftRepository.SaveAsync(...)
```

makes the method callable only through:

```
EXAMPLE / FLOW
IGiftRepository
```

If class consumers naturally need direct access, normal public implementation may be simpler.

Common Mistake — Forgetting Explicit Member Access

Given:

```
C#
void IAuditable.WriteAudit()
{
}
```

this won't compile:

```
C#
// gift.WriteAudit();
```

Use:

```
C#
((IAuditable)gift)
    .WriteAudit();
```

or hold an interface reference.

Common Mistake — Conflicting Interfaces

If:

```
EXAMPLE / FLOW
Interface A
and
Interface B
```

contain the same member name but require different semantics, one public method may accidentally satisfy both.

Use explicit interface implementation when separate behaviors are required.

Common Mistake — Default Interface Method Becomes Class Method

It does not automatically appear on the concrete class's normal public surface.

Call through the interface reference unless the class declares its own corresponding public member.

Common Mistake — Using Abstract Class for Capabilities

Suppose:

EXAMPLE / FLOW

```
Gift

Campaign

Task

User
```

all need audit behavior.

Don't create:

EXAMPLE / FLOW

```
AuditableBaseClass
```

solely because they share one capability if they otherwise belong to unrelated hierarchies.

An interface such as:

EXAMPLE / FLOW

```
IAuditable
```

may be cleaner.

Common Mistake — Using Interface for Shared Mutable State

If implementations genuinely require:

EXAMPLE / FLOW

```
Same state

Same constructor validation

Same protected lifecycle
```

forcing all of it into default interface methods can create awkward design.

An abstract base class may fit better.

Common Mistake — Ignoring Contract Meaning

An implementation should respect the interface's semantics.

If:

C#

```
Task SaveAsync(...)
```

in one implementation saves data,

but another implementation:

EXAMPLE / FLOW

```
silently deletes data
```

then the abstraction is misleading.

Polymorphism requires behavioral substitutability, not merely matching m

Common Mistake — Exposing Concrete Type from Abstraction

Suppose:

```
C#
public interface IPaymentProcessor
{
    StripePaymentResult
        Process(...);
}
```

Now the contract is tied directly to one provider.

A more abstract contract might return:

```
C#
PaymentResult
```

if that better represents the application model.

Common Mistake — Abstraction Over Every Utility

Don't replace:

```
C#
Math.Round(...)
```

with:

```
EXAMPLE / FLOW
IMathRounder

DefaultMathRounder
```

unless genuine variability/boundary needs exist.

Abstraction has a maintenance cost.

Performance Considerations

Interfaces and abstract members often involve polymorphic dispatch.

Compared with a simple directly known method call, there may be an additional runtime dispatch step.

For ordinary ASP.NET Core applications, this is usually insignificant compared with:

```
EXAMPLE / FLOW
Database access

HTTP calls

Disk I/O

Serialization
```

```
Network latency
```

Design clarity should normally come first.

Value Type Interface Boxing

Structs can implement interfaces.

Example:

```
C#
public struct GiftId :
    IComparable<GiftId>
{
}
```

If a value type is converted to an interface reference, boxing can occur in relevant scenarios:

```
EXAMPLE / FLOW
Struct value
    →
Boxed object/interface representation
```

This can matter in performance-sensitive loops.

Generics often help avoid unnecessary boxing.

You'll study this more in Topic 17 and Topic 20.

Abstract Class Allocation

Using an abstract base reference:

```
C#
FinancialTransaction transaction = new GiftTransaction(...);
```

does not create:

```
EXAMPLE / FLOW
one FinancialTransaction object
+
one GiftTransaction object
```

There is one derived object containing its base-class portion.

Interface Reference Does Not Automatically Create Another Object

```
C#
IGiftRepository repository = sqlRepository;
```

normally copies a reference.

It doesn't clone the repository object.

Do Not Optimize Away Useful Abstractions Prematurely

Don't replace a good contract with direct infrastructure coupling because:

EXAMPLE / FLOW

```
Interface call might be slower.
```

In business applications, maintainability and architectural decoupling are usually much more significant.

Measure actual performance before redesigning.

Security Considerations

Abstraction can help security by limiting what callers can do.

For example:

C#

```
public interface ISecretValidator
{
    bool Validate(
        string suppliedValue);
}
```

Callers don't receive:

EXAMPLE / FLOW

```
raw secret

encryption key

database credentials
```

They receive a narrow capability.

Capability-Oriented Security

Instead of exposing:

C#

```
public string ApiKey
{
    get;
}
```

consider exposing only the operation that requires the secret.

Conceptually:

EXAMPLE / FLOW

```
Caller
    →
IExternalApiClient.SendAsync()
```

rather than:

EXAMPLE / FLOW

```
Caller
    →
Give me API key
    →
I'll send the request myself
```

Abstraction can reduce secret exposure.

Do Not Trust Implementations Automatically

An interface such as:

EXAMPLE / FLOW

```
IAuthorizationProvider
```

defines a contract.

It doesn't prove an arbitrary implementation is trustworthy.

Security-sensitive implementations still need:

EXAMPLE / FLOW

```
Trusted registration

Correct configuration

Authorization policies

Testing

Review
```

Tenant Isolation

Good:

C#

```
public interface ITenantContext
{
    long TenantId
    {
        get;
    }
}
```

Consumers may only need:

EXAMPLE / FLOW

```
current tenant identity
```

They don't necessarily need:

EXAMPLE / FLOW

```
ability to mutate tenant
```

A narrow abstraction reduces accidental security-sensitive state modification.

Avoid Writable Security Contracts

Potentially dangerous:

C#

```
public interface IUserSecurityContext
{
    bool IsAdministrator
    {
        get;
        set;
```

```
    }
}
```


Callers can mutate authorization state.

Usually security context contracts should expose observations/operations rather than unrestricted writable privileges.

Practical Exercise — Abstract Class

Create:

```C#
public abstract class Notification
{
    protected Notification(
        string recipient)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(
            recipient);

        Recipient = recipient;
    }

    public string Recipient
    {
        get;
    }

    public abstract void Send();
}
```

Then:

```C#
public sealed class EmailNotification :
    Notification
{
    public EmailNotification(
        string email)
        : base(
            email)
    {
    }

    public override void Send()
    {
        Console.WriteLine(
            $"Email to {Recipient}");
    }
}
```

Practical Exercise — Interface

```C#
public interface INotifier
{
    void Notify(
        string message);
}
```

Implement:


```C#
public sealed class EmailNotifier :
    INotifier
{
    public void Notify(
        string message)
    {
        Console.WriteLine(
            $"EMAIL: {message}");
    }
}
```

Practical Exercise — Multiple Interfaces

```C#
public interface IAuditable
{
    string GetAuditMessage();
}

public interface IValidatable
{
    bool IsValid();
}
```

Implement:

```C#
public sealed class Gift :
    IAuditable,
    IValidatable
{
    public decimal Amount
    {
        get;
        init;
    }

    public string GetAuditMessage()
    {
        return
            $"Gift amount: {Amount}";
    }

    public bool IsValid()
    {
        return Amount > 0m;
    }
}
```

Practical Exercise — Explicit Implementation

```C#
public interface IInternalStatus
{
    string GetStatus();
}
```

```
public interface IExternalStatus
{
    string GetStatus();
}
```


Implementation:

C#
```
public sealed class GiftStatusProvider :
    IInternalStatus,
    IExternalStatus
{
    string IInternalStatus.GetStatus()
    {
        return "Internal-Processed";
    }

    string IExternalStatus.GetStatus()
    {
        return "Completed";
    }
}
```

Calls:

C#
```
IInternalStatus internalStatus = new GiftStatusProvider();

IExternalStatus externalStatus = new GiftStatusProvider();
```

Different contracts produce different representations.

Practical Exercise — Interface Inheritance

C#
```
public interface IReadGiftRepository
{
    Task<Gift?> GetAsync(
        long giftId);
}

public interface IWriteGiftRepository
{
    Task SaveAsync(
        Gift gift);
}

public interface IGiftRepository :
    IReadGiftRepository,
    IWriteGiftRepository
{
}
```

A class implementing:

EXAMPLE / FLOW
```
IGiftRepository
```

must satisfy both base contracts.

Practical Exercise — Default Interface Implementation


```C#
public interface IGiftLabelProvider
{
    string GetLabel(
        long giftId);

    public string GetDisplayLabel(
        long giftId)
    {
        return
            $"Gift: {GetLabel(giftId)}";
    }
}
```

Implementation:

```C#
public sealed class GiftLabelProvider :
    IGiftLabelProvider
{
    public string GetLabel(
        long giftId)
    {
        return
            $"GFT-{giftId}";
    }
}
```

Call:

```C#
IGiftLabelProvider provider = new GiftLabelProvider();

Console.WriteLine(
    provider.GetDisplayLabel(
        1001));
```

Practical Exercise — Choose the Right Abstraction

Requirement:

EXAMPLE / FLOW

```
Gift and Refund share
TransactionId, TenantId,
Amount and creation logic.
```

Likely:

EXAMPLE / FLOW

```
Abstract FinancialTransaction
```

Requirement:

EXAMPLE / FLOW

```
Gift, Campaign and User
can all produce audit descriptions.
```

Likely:

EXAMPLE / FLOW

```
IAuditable
```

ABSTRACTION / INTERFACES

because these types aren't necessarily one class hierarchy.

Practical Exercise — Dependency Injection

Contract:

C#

```
public interface IGiftRepository
{
    Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Service:

C#

```
public sealed class GiftService
{
    private readonly IGiftRepository
        _repository;

    public GiftService(
        IGiftRepository repository)
    {
        _repository = repository;
    }
}
```

Question:

Why depend on `IGiftRepository`?

Answer:

EXAMPLE / FLOW

```
GiftService needs persistence capability,
not knowledge of SQL implementation.
```

Interview Questions

1. What is abstraction in C#?

Abstraction means exposing essential operations or contracts while hiding implementation details that callers don't need.

Example:

EXAMPLE / FLOW

```
IGiftRepository.SaveAsync()
```

exposes persistence capability without exposing SQL details.

2. What is the difference between abstraction and encapsulation?

Abstraction focuses on:

EXAMPLE / FLOW

```
WHAT a component provides.
```

Encapsulation focuses on:

EXAMPLE / FLOW

```
HOW internal state and implementation
are protected.
```

Interfaces and abstract classes often provide abstraction.

Private members and controlled state changes support encapsulation.

3. What is an abstract class?

An abstract class is a class that cannot be instantiated directly and is intended to act as a base for derived classes.

It can contain both abstract and concrete members.

4. Does an abstract class need an abstract method?

No.

A class can be declared abstract even when all of its members have implementations.

5. Can an abstract class be instantiated?

No.

You instantiate a concrete derived class instead.

6. Can an abstract class have constructors?

Yes.

Its constructors initialize the base-class portion of derived objects.

They are often `protected`.

7. Are constructors abstract?

No.

Constructors aren't inherited and overridden like abstract methods.

8. What is an abstract method?

A method declaration with no implementation in the abstract class that requires a concrete derived class to provide an override.

Example:

C#

```
public abstract void Process();
```

9. Is an abstract method virtual?

Conceptually yes: abstract members participate in virtual dispatch and derived classes implement them using `override`.

10. What is the difference between abstract and virtual?

`abstract`:

EXAMPLE / FLOW

```
No base implementation

Concrete derived implementation required
```

`virtual`:

EXAMPLE / FLOW

```
Base implementation exists

Derived override optional
```

11. Can properties be abstract?

Yes.

A base class can require derived classes to provide property implementations.

12. What is an interface?

An interface defines a contract that implementing classes, records, or structs must satisfy.

Modern interfaces may also provide default/static functionality.

13. Can an interface be instantiated?

No.

An interface variable refers to an object implementing that interface.

14. Can a class implement multiple interfaces?

Yes.

A class can directly derive from only one class but can implement multiple interfaces.

15. Can an interface inherit multiple interfaces?

Yes.

An interface can list multiple base interfaces.

16. Can a struct implement an interface?

Yes.

Classes, structs, and records can implement interfaces.

17. What is implicit interface implementation?

The class exposes a public member matching the interface contract.

Example:

```C#
public void Process()
{
}
```

This member can normally be called through both the class and interface references.

18. What is explicit interface implementation?

A member is implemented with the interface-qualified name:

```C#
void IProcessor.Process()
{
}
```

It can normally be invoked only through an `IProcessor` reference, not directly through the concrete class.

19. Why use explicit interface implementation?

Typical reasons:

EXAMPLE / FLOW

```
Resolve conflicting interface members

Provide different behavior for
two interfaces

Hide interface-specific operation
from class's normal public surface
```

20. Does an explicit interface implementation have `public`?

No.

It doesn't declare a normal access modifier. Its accessibility comes through the interface.

21. If two interfaces declare the same method, can one class method implement both?

Yes.

One compatible public implementation can satisfy both interface mappings.

If different behavior is needed, explicit interface implementation can separate them.

22. Can an interface contain methods with bodies?

Yes in modern C#.

These are default interface implementations.

23. Why were default interface methods introduced?

One important purpose is allowing published interfaces to evolve by adding functionality with a default implementation rather than immediately breaking every existing implementing type.

24. Does a default interface method become a normal method on the implementing class?

Not automatically.

If the class doesn't itself declare the method, the default member is accessed through an interface reference.

25. Can interfaces contain fields?

Modern interfaces may contain **static fields**, but they don't provide ordinary per-instance fields for implementing objects.

For shared instance state, an abstract class is usually the natural mechanism.

26. Can interfaces contain properties?

Yes.

A normal interface property declaration defines a required property contract rather than creating an auto-property storage field.

27. Can interfaces contain events and indexers?

Yes.

Modern C# interface declarations may contain methods, properties, indexers, events, operators, static members, nested types, and other supported members.

28. What is `static abstract` in an interface?

It declares a static operation that implementing types must provide.

This enables type-level generic contracts such as generic math or factory-like behavior.

29. How are static abstract interface members normally called generically?

Through a type parameter constrained to the interface.

Example:

C#

```
static T Create<T>()
    where T : IFactory<T>
{
    return T.Create();
}
```

30. What is `static virtual` in an interface?

It allows an interface to supply default static behavior that implementing types can specialize.

It is primarily an advanced generic-programming feature.

31. What is the biggest difference between abstract class and interface?

A useful summary:

EXAMPLE / FLOW

```
Abstract class
    →
Shared base identity/state/implementation

Interface
    →
Shared contract/capability
across potentially unrelated types
```

32. Can an abstract class implement an interface?

Yes.

It can implement the interface concretely or map interface requirements to abstract members that later concrete derived classes implement.

33. When should an abstract class be preferred?

When related types share:

EXAMPLE / FLOW

```
IS-A relationship

Instance state

Constructor logic

Concrete implementation

Protected extension behavior
```

34. When should an interface be preferred?

When consumers require:

EXAMPLE / FLOW

```
A capability/contract

Multiple implementations

Dependency inversion

Testing substitution

Cross-hierarchy behavior
```

35. What is interface polymorphism?

A caller operates through an interface reference while different implementing runtime types provide behavior.

Example:

C#

```
IGiftRepository repository = new SqlGiftRepository();
```

later:

C#

```
repository = new FakeGiftRepository();
```

The caller still uses the same interface contract.

36. Why are interfaces useful with dependency injection?

They let services depend on contracts rather than concrete infrastructure.

Example:

EXAMPLE / FLOW

```
GiftService
    →
IGiftRepository
```

instead of:

EXAMPLE / FLOW

```
GiftService
    →
SqlGiftRepository
```

The DI container provides the implementation.

37. Does every service need an interface?

No.

Create an interface when it represents a useful abstraction boundary, variability point, testing substitution, or external/infrastructure contract.

An interface merely mirroring every class without purpose adds complexity.

38. What is a fat interface?

An interface with too many unrelated responsibilities.

Example:

EXAMPLE / FLOW

```
Save
Email
Charge
Authorize
Export
Print
Audit
```

all in one interface.

Smaller cohesive contracts are generally easier to implement and maintain.

39. What is interface inheritance?

An interface can combine or extend contracts from other interfaces.

Example:

C#

```
interface IGiftRepository :
    IGiftReader,
    IGiftWriter
```

Implementing `IGiftRepository` means satisfying its inherited contracts too.

40. Can interface calls cause boxing?

When a value type is converted to an interface reference, boxing may occur in applicable scenarios.

This matters primarily in performance-sensitive value-type code.

Classes don't have that particular boxing concern.

Best Practices

Use abstraction to expose:

```
EXAMPLE / FLOW
WHAT consumers need
```

without unnecessarily exposing:

```
EXAMPLE / FLOW
HOW it is implemented.
```

Use abstract classes when:

```
EXAMPLE / FLOW
Related types share real IS-A semantics

Shared state exists

Shared construction exists

Shared implementation is useful
```

Use interfaces when:

```
EXAMPLE / FLOW
You need a capability or contract

Multiple implementations are useful

Different class hierarchies need
the same behavior

You want an infrastructure boundary
```

Keep interfaces cohesive.

Prefer narrow contracts.

Don't expose implementation technology unnecessarily.

Prefer:

```
C#
IGiftRepository
```

over:

```
C#
ISqlGiftRepository
```

when SQL is supposed to be hidden.

Use normal public interface implementation for ordinary APIs.

Use explicit implementation when:

```
EXAMPLE / FLOW
Contracts conflict

Different interface behavior is needed
```

```
Member shouldn't appear
on concrete class surface
```


Remember:

EXAMPLE / FLOW

```
Explicit implementation
    →
call through interface reference.
```

Use default interface implementations carefully.

They are especially useful for:

EXAMPLE / FLOW

```
contract evolution
```

but aren't a general replacement for abstract classes.

Remember default interface members do not automatically become normal members of the implementing class.

Use `static abstract` only when generic type-level behavior is genuinely needed.

Don't introduce advanced interface features merely because they exist.

Use dependency injection to connect:

EXAMPLE / FLOW

```
Abstraction
    →
Implementation
```

rather than manually constructing infrastructure deep inside business services.

Don't create an interface for every class automatically.

Don't create giant "everything" interfaces.

Keep authorization and secret handling behind narrow abstractions.

Avoid writable security-sensitive contracts.

Use abstraction to make testing easier, but don't over-engineer simple deterministic functionality.

Quick Revision

Concept	Meaning
Abstraction	Expose essential behavior, hide implementation detail
Encapsulation	Protect/control internal state
Abstract class	Non-instantiable base class
Abstract method	Derived concrete type must implement
Abstract property	Property contract implemented by derived type
`override`	Implements inherited abstract/virtual behavior
Interface	Contract/capability
Interface implementation	Type satisfies interface members

Concept	Meaning
Implicit implementation	Normal public matching member
Explicit implementation	Member accessible through interface reference
Multiple interfaces	Type implements several contracts
Interface inheritance	One interface extends other interfaces
Default interface method	Interface supplies implementation
`static abstract`	Required static member on implementing type
`static virtual`	Static interface member with default specialization capability
Abstract class state	Supported
Interface instance field state	Not ordinary interface behavior
DI abstraction	Consumer depends on contract
Substitutability	Implementations respect same contract

Complete Fundraising Example

We can now combine abstract classes and interfaces into a realistic model.

Domain Abstraction

C#

```
public interface ITenantScoped
{
    long TenantId
    {
        get;
    }
}
```

C#

```
public interface IAuditable
{
    string GetAuditDescription();
}
```

Abstract Transaction

C#

```
public abstract class FinancialTransaction :
    ITenantScoped,
    IAuditable
{
    protected FinancialTransaction(
        long transactionId,
        long tenantId,
        decimal amount,
        DateTime createdAtUtc)
    {
        if (transactionId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(transactionId));
```

```
        }

        if (tenantId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(tenantId));
        }

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        TransactionId = transactionId;

        TenantId = tenantId;

        Amount = amount;

        CreatedAtUtc = createdAtUtc;
    }

    public long TransactionId
    {
        get;
    }

    public long TenantId
    {
        get;
    }

    public decimal Amount
    {
        get;
    }

    public DateTime CreatedAtUtc
    {
        get;
    }

    public string GetBaseSummary()
    {
        return
            $"Transaction {TransactionId}: "
            +
            $"{Amount:C2}";
    }

    public abstract string
        GetAuditDescription();
}
```

Gift Transaction

C#

```
public sealed class GiftTransaction :
    FinancialTransaction
{
    public GiftTransaction(
        long transactionId,
```

```
        long tenantId,
        long constituentId,
        decimal amount,
        long? campaignId,
        DateTime createdAtUtc)
        : base(
            transactionId,
            tenantId,
            amount,
            createdAtUtc)
    {
        if (constituentId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(constituentId));
        }

        ConstituentId = constituentId;

        CampaignId = campaignId;
    }

    public long ConstituentId
    {
        get;
    }

    public long? CampaignId
    {
        get;
    }

    public override string
        GetAuditDescription()
    {
        return
            $"Gift transaction "
            +
            $"{TransactionId} "
            +
            $"for constituent "
            +
            $"{ConstituentId}.";
    }
}
```

Refund Transaction

C#

```
public sealed class RefundTransaction :
    FinancialTransaction
{
    public RefundTransaction(
        long transactionId,
        long tenantId,
        long originalGiftId,
        decimal amount,
        DateTime createdAtUtc)
        : base(
            transactionId,
            tenantId,
            amount,
            createdAtUtc)
```

```
    {
        if (originalGiftId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(originalGiftId));
        }

        OriginalGiftId = originalGiftId;
    }

    public long OriginalGiftId
    {
        get;
    }

    public override string
        GetAuditDescription()
    {
        return
            $"Refund transaction "
            +
            $"{TransactionId} "
            +
            $"for gift "
            +
            $"{OriginalGiftId}.";
    }
}
```

Repository Abstraction

C#

```
public interface IGiftRepository
{
    Task<Gift?> GetAsync(
        long giftId,
        CancellationToken cancellationToken);

    Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Infrastructure Implementation

C#

```
public sealed class SqlGiftRepository :
    IGiftRepository
{
    public async Task<Gift?> GetAsync(
        long giftId,
        CancellationToken cancellationToken)
    {
        // EF Core / SQL implementation.

        await Task.CompletedTask;

        return null;
    }

    public async Task SaveAsync(
```

```
        Gift gift,
        CancellationToken cancellationToken)
    {
        // EF Core / SQL implementation.

        await Task.CompletedTask;
    }
}
```

Test Implementation

C#

```
public sealed class InMemoryGiftRepository :
    IGiftRepository
{
    private readonly Dictionary<long, Gift>
        _gifts = [];

    public Task<Gift?> GetAsync(
        long giftId,
        CancellationToken cancellationToken)
    {
        _gifts.TryGetValue(
            giftId,
            out Gift? gift);

        return Task.FromResult(
            gift);
    }

    public Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        _gifts[gift.GiftId] = gift;

        return Task.CompletedTask;
    }
}
```

Receipt Abstraction

C#

```
public interface IGiftReceiptSender
{
    Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Email implementation:

C#

```
public sealed class EmailGiftReceiptSender :
    IGiftReceiptSender
{
    public Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
```

```
        Console.WriteLine(
            $"Email receipt for Gift "
            +
            $"{gift.GiftId}.");

        return Task.CompletedTask;
    }
}
```

Application Service

C#

```
public sealed class GiftService
{
    private readonly IGiftRepository
        _repository;

    private readonly IGiftReceiptSender
        _receiptSender;

    public GiftService(
        IGiftRepository repository,
        IGiftReceiptSender receiptSender)
    {
        ArgumentNullException.ThrowIfNull(
            repository);

        ArgumentNullException.ThrowIfNull(
            receiptSender);

        _repository = repository;

        _receiptSender = receiptSender;
    }

    public async Task ProcessAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        ArgumentNullException.ThrowIfNull(
            gift);

        await _repository.SaveAsync(
            gift,
            cancellationToken);

        await _receiptSender.SendAsync(
            gift,
            cancellationToken);
    }
}
```

Architecture Flow

EXAMPLE / FLOW

```
GiftsController
        ↓
GiftService
        ↓
        |
  ┌─────┴──────────────────┐
  ↓                        ↓
```

```
IGiftRepository   IGiftReceiptSender
      ↓                   ↓
SqlGiftRepository EmailGiftReceiptSender
      ↓
SQL Server
```

The service doesn't need to know:

EXAMPLE / FLOW

```
How SQL works

How email is sent

Which provider is used

How tests persist gifts
```

It knows only:

EXAMPLE / FLOW

```
Required Contracts
```

That is abstraction.

Abstract Class Flow

EXAMPLE / FLOW

```
FinancialTransaction
        ↓
Shared State

TransactionId
TenantId
Amount
CreatedAtUtc
        ↓
Shared Behavior

GetBaseSummary()
        ↓
Required Specialized Behavior

GetAuditDescription()
        ↓
  ┌─────┴─────┐
  ↓           ↓
GiftTransaction RefundTransaction
```

Interface Flow

EXAMPLE / FLOW

```
IGiftRepository
        ↓
Contract
        ↓
  ┌─────┴──────────┐
  ↓                ↓
SqlGiftRepository  InMemoryGiftRepository
```

Caller:

EXAMPLE / FLOW

```
GiftService
```

sees only:

EXAMPLE / FLOW

```
IGiftRepository
```

Abstract Class vs Interface Decision Flow

EXAMPLE / FLOW

```
Need abstraction?
    ↓
Do types share
true IS-A relationship?
    ↓
Yes
    ↓
Do they also share
state / constructor /
implementation?
    ↓
Yes
    ↓
Abstract Class
```

Otherwise:

EXAMPLE / FLOW

```
Do unrelated or differently
inherited types share
a capability/contract?
    ↓
Yes
    ↓
Interface
```

Multiple Capabilities

EXAMPLE / FLOW

```
Gift
    ↓
ITenantScoped

IAuditable

IValidatable
```

This is natural because interfaces represent capabilities.

Shared Stateful Hierarchy

EXAMPLE / FLOW

```
FinancialTransaction
    ↓
GiftTransaction
```

```
RefundTransaction
```


This is natural because:

EXAMPLE / FLOW

```
GiftTransaction
IS A
FinancialTransaction
```

and they share state/implementation.

Final End-to-End Mental Model

EXAMPLE / FLOW

```
Abstraction
    →
Hide unnecessary details
    →
Expose essential contract
```

Two major tools:

EXAMPLE / FLOW

```
Abstract Class
    →
Shared hierarchy
+
shared state
+
shared implementation
+
required specialization
```

and:

EXAMPLE / FLOW

```
Interface
    →
Contract
+
capability
+
multiple implementations
+
multiple interface support
```

Implementation:

EXAMPLE / FLOW

```
Interface
    →
Concrete Class
    →
Actual behavior
```

Runtime:

EXAMPLE / FLOW

```
Caller
    →
Abstraction Reference
    →
Concrete Runtime Object
    →
Polymorphic Implementation
```

Architecture:

EXAMPLE / FLOW

```
Controller
    →
Service Abstraction
    →
Business Implementation
    →
Repository Abstraction
    →
Infrastructure Implementation
```

The core lesson is:

> **Abstraction lets higher-level code express what it needs without depending on low-level implementation details. Use an abstract class when related types share a genuine inheritance hierarchy, state, construction, and implementation. Use an interface when you need a capability or contract that can be satisfied by multiple implementations—possibly across unrelated class hierarchies.**

In your Fundraising Platform, this distinction leads naturally to designs such as:

EXAMPLE / FLOW

```
FinancialTransaction
    →
Abstract class
```

because gift and refund transactions may share state and identity,

while:

EXAMPLE / FLOW

```
IGiftRepository

IGiftReceiptSender

IPaymentProcessor

IAuditable
```

are strong interface candidates because they describe capabilities and replaceable contracts.

Next roadmap topic: Topic 16 — Composition, Association, and Object Relationships.